



Do requisito ao sistema

Aula 15 · Bloco 4 — Java Moderno

Do requisito ao diagrama, do diagrama ao commit final

Nesta aula

Você não aprende conceito novo: você **usa todos**.

1. Escolher o **tema** — e o escopo
2. Do **requisito** ao **diagrama**
3. Estratégia de **commits**
4. Revisão de código de um colega

A régua: 5 classes em `model` / `service` / `app`, abstrata + interface, polimorfismo sem `if` de tipo, exceção própria, persistência e stream.

Um bom tema tem três coisas

- **2 a 4 entidades** relacionadas;
- Uma **hierarquia natural** — algo que se divide em tipos;
- **Regras de negócio de verdade** — limites, cálculos, estados que mudam.

Cinco temas que funcionam

Tema	Hierarquia	A regra que dá sal
Academia	Plano → mensal, anual	bloquear check-in vencido
Lanchonete	ItemCardapio → lanche, combo	combo com desconto
Locadora	Midia → físico, digital	multa por atraso
Clínica	Consulta → 1ª vez, retorno	sem choque de horário
Estacionamento	Veiculo → carro, moto	hora + tolerância de 15 min

O erro nº 1 é escolher grande demais

"Uma rede social completa" não termina.

Prefira **um** fluxo bem feito:

cadastrar → operar → consultar → relatório

Terminado o básico, você acrescenta extras à vontade.

Do requisito ao diagrama, em 5 passos

1. **Escreva 5 frases** sobre o que o sistema faz — em português, do ponto de vista de quem usa;
2. **Sublinhe os substantivos** → candidatos a classe. **Circule os verbos** → candidatos a método;
3. Pergunte "**é um**" ou "**tem um**" em cada par: **é um** vira herança, **tem um** vira atributo;
4. Procure a **capacidade transversal** — algo que classes sem parentesco fazem → vira **interface**;
5. **Desenhe** em Mermaid e ponha no README **antes** de programar.

O percurso, num exemplo

*"O sistema registra a **entrada** de um **veículo**, calcula o **valor** na **saída** conforme o **tipo** e o **tempo**, e emite um **relatório** do dia."*

- **Substantivos** → `Veiculo`, `Ticket`, `Estacionamento`, `Relatorio`;
- `Carro` é um `Veiculo` → herança. `Ticket` tem um `Veiculo` → composição;
- **Verbos** → `registrarEntrada`, `registrarSaida`, `calcularValor`;
- `calcularValor()` diferente por tipo → método **abstrato** em `Veiculo`.

A sequência de commits que funciona

- | | |
|----------------------------------|----------------------------|
| 1. Estrutura de pastas e pacotes | 7. Exceções personalizadas |
| 2. Classes de modelo | 8. Menu básico funcionando |
| 3. Encapsulamento e validações | 9. Persistência em arquivo |
| 4. Herança/interface do domínio | 10. Relatório com streams |
| 5. Service: cadastro e busca | 11. README com diagrama |
| 6. Service: a operação principal | 12. Ajustes finais |

Doze commits, cada um deixando o programa **rodando**.

A regra prática do commit

Se você não consegue descrever o commit
em uma frase curta,
ele está grande demais.

E um projeto entregue num commit gigante na véspera
não conta história nenhuma — o `git log` é parte da
entrega.

Antes de sair da aula

- ☐ Tema escolhido, com escopo **pequeno o bastante para terminar**;
- ☐ As 5 frases escritas e os substantivos sublinhados;
- ☐ Diagrama de classes em Mermaid, no README;
- ☐ Repositório criado, com os primeiros commits.

Próxima aula

Aula 16 — Revisão e Próximos Passos

O mapa do curso, os quatro pilares num exemplo só,
e para onde ir depois.